- **设计选择（Design choices）**：指在构建模型过程中，从数据处理到模型架构搭建的一系列决策。在数据处理方面，包括对输入数据的处理方式，如是否进行归一化，像文中提到的将输入数据通过公式$x \leftarrow \frac{x-\mu}{\sigma^2}$进行归一化处理，以提升训练效果；在模型架构上，选择使用何种神经网络结构，如全连接神经网络、卷积神经网络或循环神经网络等，不同结构适用于不同类型的数据和任务。
- **超参数调整（Hyperparameter tuning）**：超参数是在模型训练前需要人为设定的参数，无法通过模型训练直接学习得到。例如学习率，它决定了模型在训练过程中参数更新的步长。合适的学习率对模型训练至关重要，学习率过小会使训练速度缓慢，还可能陷入局部最优解；学习率过大则可能导致训练过程不稳定，错过最优解。除学习率外，还有批处理大小、神经网络的层数和神经元数量等都是超参数。调整超参数的方法有试错法、网格搜索、随机搜索等，旨在找到一组最优超参数，使模型性能达到最佳。

UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lectutorial 3: Design choices & Hyperparameter tuning

Normalisation, Learning rate, Faster Optimisers & Hyperparameter tuning

Instructor: Thomas Lacombe

归一化、学习率、更快的优化器与超参数调优

Learning outcomes

- Understand the effect of input normalisation on neural network training.
- Understand Batch Normalisation principles and effects on training
- Understand several possible approaches to tuning the learning rate
- Understand different optimisation algorithms based on adaptive learning rate (Momentum, AdaGrad, RMSProp, ADAM)

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

Extra resources

If you want to go further:

- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks, chapters 7, 8 and 11.

- *Hands-on Machine Learning with Scikit-Learn, Keras and Tensorflow*, Géron A.
  - Chapters 10 and 11.

Activation functions, weights initialisation & regularisation

激活函数、权重初始化、正则化
“ Activation functions, weights initialisation & regularisation ”
意为“ 激活函数、权重初始化与正则化 ”，是神经网络中的重要概念：
- **激活函数（Activation functions）**：
激活函数为神经网络引入非线性因素，使网络能够学习复杂的模式。若没有激活函数，多层神经网络就等价于一个线性模型，其表达能力会非常有限。常见的激活函数有Sigmoid函数、ReLU函数等。Sigmoid函数将输入映射到(0, 1)区间，公式为$\sigma(x)=\frac{1}{1 + e^{-x}}$ ，常用于二分类问题的输出层；ReLU函数表达式为f(x)=max(0, x)，计算简单，能有效缓解梯度消失问题，在隐藏层中应用广泛。

- **权重初始化（weights initialisation）**：
在神经网络训练开始前，需要对权重进行初始化。合适的初始化方式对模型的训练和性能至关重要。若权重都初始化为0，会导致神经元在训练过程中学习到相同的特征，破坏网络的对称性，无法有效学习。常用的初始化方法有随机初始化，如使用正态分布或均匀分布来生成初始权重；还有Xavier初始化、Kaiming初始化等，这些方法能根据网络的结构和激活函数的特性，合理地初始化权重，使信号在网络中更有效地传播，加快收敛速度。

- **正则化（regularisation）**：正则化是防止神经网络过拟合的重要手段。过拟合时，模型在训练集上表现很好，但在测试集或新数据上表现很差。L1和L2正则化是常见的方法，L2正则化也叫权重衰减，通过在损失函数中添加权重向量的L2范数（平方和的平方根）的惩罚项，使权重值趋向于更小，避免权重过大导致模型过于复杂，增强模型的泛化能力。此外，还有Dropout正则化，它在训练过程中随机“ 丢弃 ”一些神经元及其连接，减少神经元之间的共适应性，迫使网络学习更鲁棒的特征。

Normalising inputs

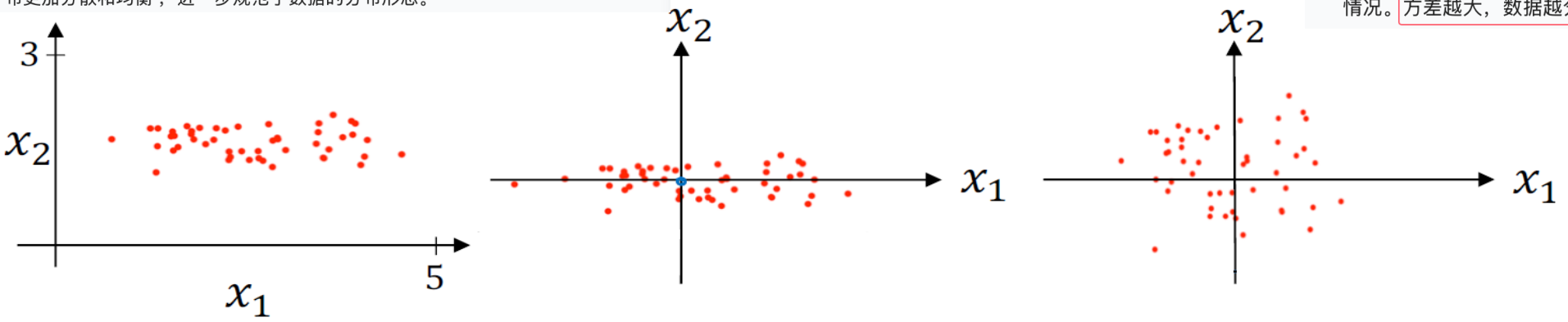
Normalising inputs of a NN:

这三张图展示了数据在不同状态下的分布情况，可能用于说明数据归一化等操作的效果。

从左到右推测：

- 第一张图是原始的二维数据分布，数据点在x_1和x_2构成的平面上分散分布，x_1轴大概范围到 5 ，x_2轴大概到 3 。
- 第二张图可能是对数据进行了中心化操作，数据点围绕原点聚集，说明数据的均值被调整到了原点附近。
- 第三张图可能是在中心化基础上，还进行了一定的尺度变换之类的处理，数据分布更加分散和均衡 ，进一步规范了数据的分布形态。

$$x \leftarrow \frac{x - \mu}{\sigma^2}$$



“Normalising inputs” 即输入归一化，是对神经网络输入数据进行预处理的一种关键操作，公式为$x \leftarrow \frac{x-\mu}{\sigma^2}$ ，在神经网络训练过程中具有重要意义：

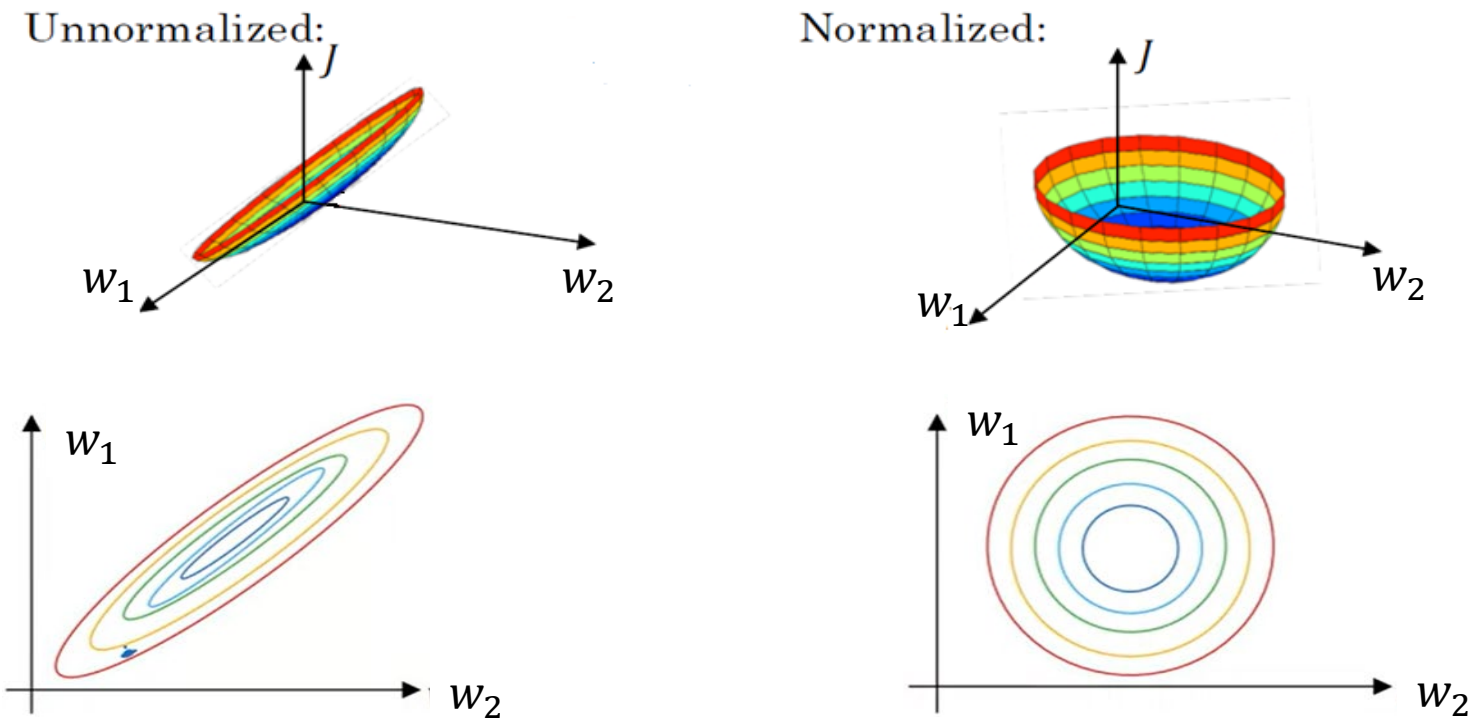
- **加快训练速度**：归一化使输入数据处于相似的尺度，避免某些特征因数值过大或过小，导致模型训练时参数更新的步长不一致。例如，在训练图像识别模型时，若图像像素值未归一化，不同图像的像素值范围差异大，模型在学习时难以快速收敛。而经过归一化后，模型能更高效地学习特征，加快训练进程 [1] 。
- **降低梯度消失 / 爆炸风险**：在神经网络中，梯度消失或爆炸会使训练无法正常进行。当输入数据未归一化时，各层输入数据的分布差异大，经过多层网络传递后，梯度可能会变得极小（梯度消失）或极大（梯度爆炸） 。输入归一化可使各层输入数据分布相对稳定，降低这种风险，保证训练的稳定性和有效性 [2] 。

符号含义

- x：表示原始数据中的某个样本或特征值。在实际应用场景中，它可以是图像的像素值、文本数据的特征表示、传感器采集的数据值等。
- μ：代表数据的均值，即所有样本数据的平均值。计算均值是为了衡量数据的中心位置。
- σ^2：表示数据的方差，用于衡量数据的离散程度，即数据的分布范围和波动情况。方差越大，数据越分散；方差越小，数据越集中。

Normalising inputs

- **Why normalising inputs?**



→Makes training faster.

→Reduces risk of vanishing/exploding gradients.

Images source: Andrew Ng

Batch normalisation

- Batch norm = normalising the input of each layer to enable faster training in each layer.

1. Computes the mean of the layer's inputs: μ
2. Computes the variance of the layer's inputs: σ^2
3. Compute the normed input value of each neuron in the layer:

$$z_{norm} = \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

4. Compute: $\tilde{z} = \gamma * z_{norm} + \beta$ (γ and β are learnable parameters)

- Running mean and variance are calculated during training and used at prediction time.

- **符号含义**：
  - z_{norm}：表示经过归一化后的输入值，即每个神经元最终得到的归一化后的输入。
  - z：代表归一化之前，该神经元原本的输入值，它可以是前一层神经元的输出经过加权求和等操作后的结果。
  - μ：是当前 mini - batch 数据中，该神经元所有输入值的均值。通过计算均值可以衡量这批数据的中心位置。
  - σ^2：为当前 mini - batch 数据中，该神经元所有输入值的方差，用于衡量数据的离散程度，方差越大说明数据越分散。
  - ε：是一个很小的正数，比如10^{-8}，引入它是为了防止分母$\sqrt{\sigma^2}$为 0 的情况，保证计算的稳定性。
- **计算过程及作用**：公式通过将原始输入值z减去均值μ，再除以方差σ^2加上一个小正数ε后的平方根，实现对神经元输入值的归一化。归一化后的数据分布更稳定，有助于减少内部协变量偏移，加快神经网络的训练速度，还能使模型在训练时更稳定，不易出现梯度消失或梯度爆炸的问题。

"Compute: $\tilde{z} = \gamma * z_{norm} + \beta$($\gamma$ nd β are learnable parameters)" 意思是进行计算：$\tilde{z}$等于可学习参数γ乘以归一化后的输入值z_{norm}，再加上可学习参数β。

公式解释

- **符号含义**：
  - $\tilde{z}$：是经过批量归一化最终得到的输出值，这个值会作为下一层的输入。
  - γ：是一个可学习的缩放参数（scale parameter），它的作用是对归一化后的数据进行缩放调整，通过训练学习到合适的值，以帮助模型更好地拟合数据。
  - z_{norm}：是之前计算得到的归一化后的输入值，通过公式$z_{norm} = \frac{z-\mu}{\sqrt{\sigma^2+\varepsilon}}$计算得出。
  - β：是一个可学习的偏移参数（shift parameter），用于对缩放后的数据进行偏移操作，同样也是在训练过程中学习到合适的值。
- **计算过程及作用**：在对神经元输入进行归一化得到z_{norm}后，直接使用这个归一化的值可能会限制模型的表达能力。通过引入可学习的参数γ和β，模型可以根据数据的特征，学习到对归一化数据的最佳缩放和偏移方式，使得模型既能享受归一化带来的稳定训练的好处，又能保持足够的表达能力来学习复杂的模式。

Batch normalisation - Regularisation effect

- Batch norm uses the mean and variance of mini-batches, and not the entire dataset.
- It introduces some noise in the neuron values, therefore in the activation values.

→This leads to some regularisation effect.

- Similar effect than using dropout.

- Also reduces the influence of weight initialisation!

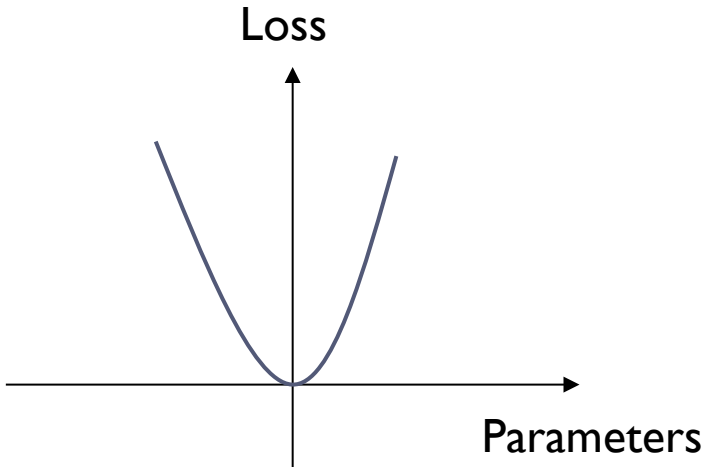
How to improve learning? Learning rate

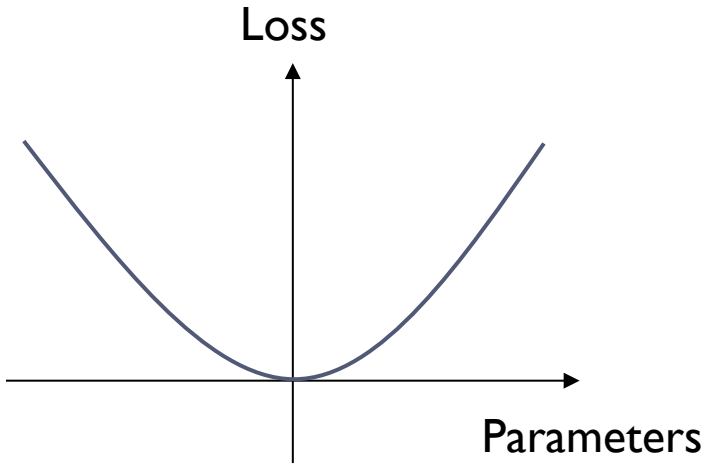
$$w \leftarrow w - \alpha * \nabla_w g(w)$$

- "One hyperparameter to rule them all."
  - Often seen as the most important hyperparameter to tune.
- Hard to know in advance what will be the best value.
  - Trial/error (common range: $10^{-6} < \alpha < 1.0$)
  - Grid/random/etc. search, optimisation techniques.

Choosing a learning rate

- Learning rate value vs curvature of the loss function





- The right learning rate depends on the slope!

- If α too small: learning will be slow and risk of local minima.
- If α too big: risk of overshooting (especially if slope is steep).

Learning rate

- **Learning rate decay schedule**
  - Common practice: decrease learning rate over time (learning rate decay).
  - E.g., linear decay.
  - Linear decay for set number of iterations and then constant.

- **Adaptive learning rates strategies**
  - Monitors the model's performance and adapt the learning rate in response.
  - Reduces learning rate when performance plateaus.
  - Increases learning rate when performance does not improve for a number of iterations.

Learning rate decay schedule

- A common practice: decaying learning rate as learning progresses.

- Why?
  - → Cost function less steep as you come close to an optimum.
  - → Decaying the learning rate allows to take smaller steps when approaching the optimum.

- Finding the right decay schedule is non-trivial.
  - Time-based: Linear decay, exponential decay?
  - Step-based: how much to drop every how many epochs?

https://towardsdatascience.com/learning-rate-schedules-and-adaptive-learning-rate-methods-for-deep-learning-2c8f433990d1
https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/

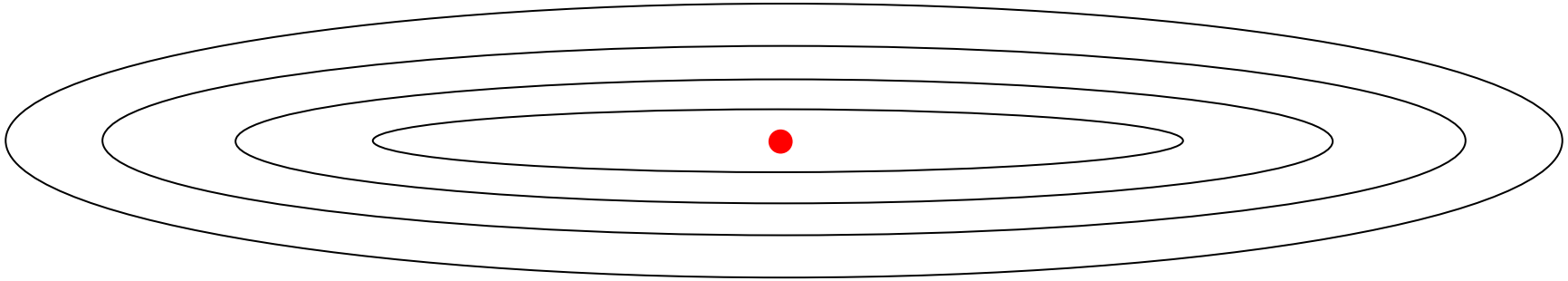
Hyperparameter tuning, batch-norm and learning rate schedule

Algorithms with adaptive learning rate

- SGD uses a fixed learning rate value.
- Other gradient optimisation algorithms implements adaptive learning rates.
- Adagrad, RMSProp, AdaDelta, ADAM, etc.
- Adaptive learning rate strategies generally outperform fixed and not well tuned learning rates.

Oscillations in learning

- **Problem**: oscillations make training slower and increase the risk of overshooting if the learning rate is too high.

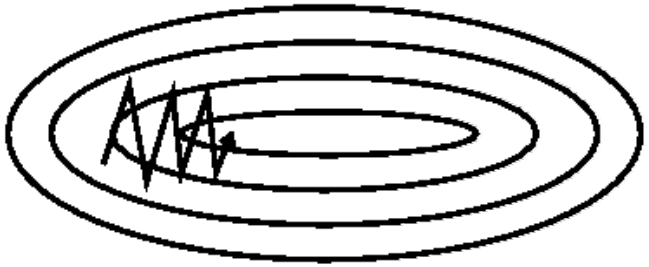


Add momentum

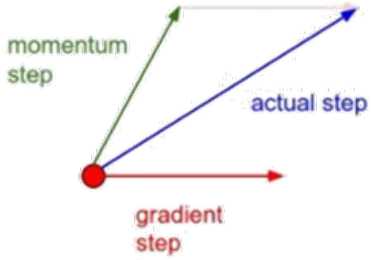
- **Momentum mechanism**: includes an exponentially decaying weighted average of previous gradients in the weight update.
  - Usually used with mini-batch Gradient Descent.

$$w \leftarrow w - \alpha \nabla_w g(w)$$

Without momentum



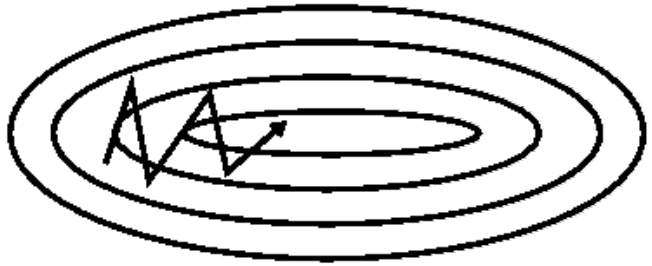
Momentum update



$$v_t = \beta v_{t-1} + (1-\beta)\nabla_w g(w)$$

$$w \leftarrow w - \alpha v_t$$

With momentum



Images source : https://cs231n.github.io/neural-networks-3/, https://www.willamette.edu/~gorr/classes/cs449/momrate.html

AdaGrad

- **Ada**ptive (sub)**Grad**ients algorithm: decays the learning rate based on the sum of previous squared gradients for each individual weight.

$$v_t = v_{t-1} + (\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

- Adapts the learning rate for each variable/dimension (i.e., weight).
- Reduces the influence of weights with high gradients and increase the influence of weights with lesser gradients (infrequent updates).

Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization.

AdaGrad

- **Problem with Adagrad**: it is very slow and could stop before reaching the minimum!

- Caused by sum of squared gradients monotonically increasing.

$$v_t = v_{t-1} + (\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

RMSProp

- **R**oot **M**ean **S**quared **Prop**agation algorithm: extension of Adagrad, introducing a decaying sum of squared gradients.

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

- Keeps the sum of squared gradients under control by applying a decay factor.
- RMSProp does not suffer from problem mentioned for Adagrad.

Lecture 6e rmsprop: Divide the gradient by a running average of its recent magnitude, Geoffrey Hinton

ADAM

- **Adaptive Moment Estimation** algorithm: combines momentum and RMSprop to go even faster!

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_w g(w) \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_w g(w))^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

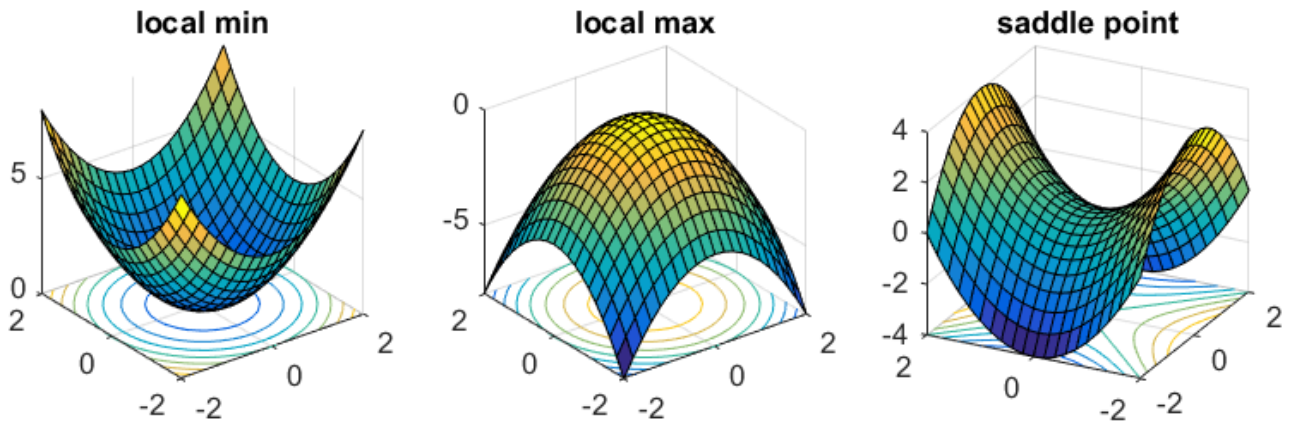
$$w \leftarrow w - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}}\hat{m}_t$$

- ADAM usually achieves good result fast on a good range of tasks → safe as a first choice.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization.
Coursera video about bias correction

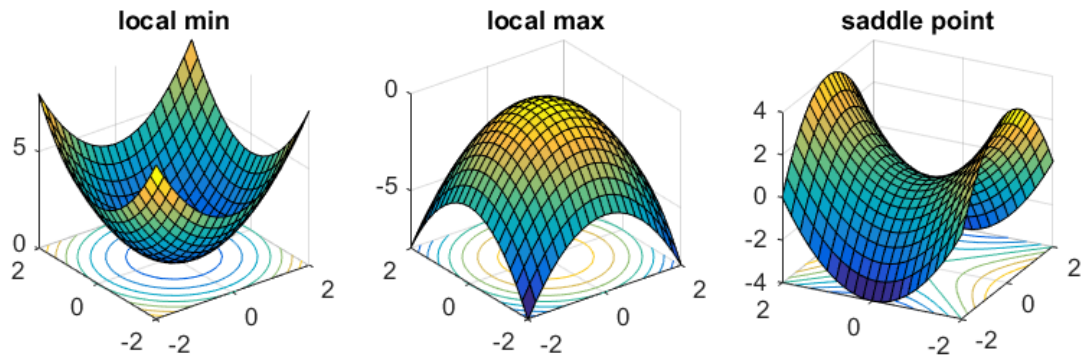
When are ADAM & co. useful?

- **Saddle points / Plateaus**



Local optima?

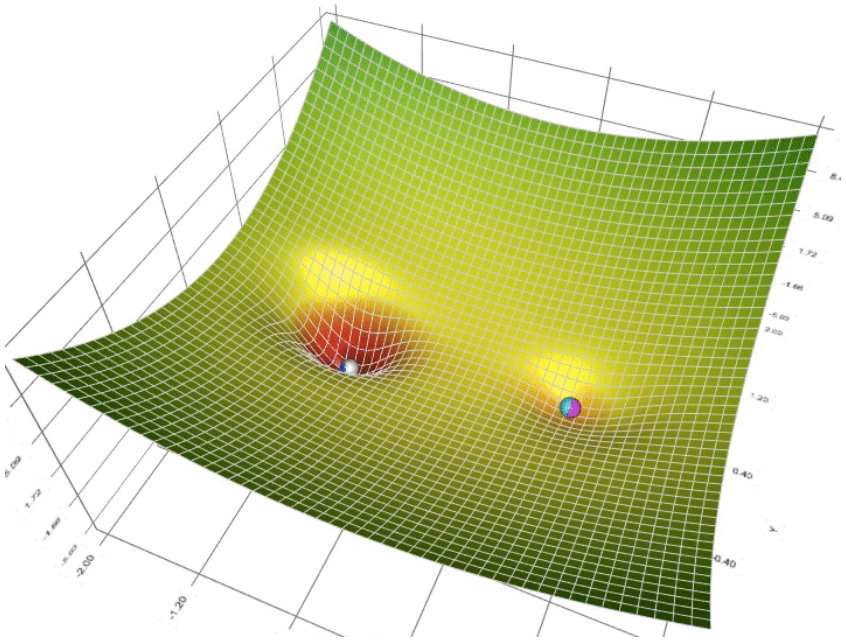
- Not likely to encounter a local optima with a large NN.
- High number of parameters → high number of dimensions for the cost function.
- A local optima (e.g., minima) require convex/concave shape in every dimension, which is very unlikely.
- More likely to get saddle points.



Visualisation tool

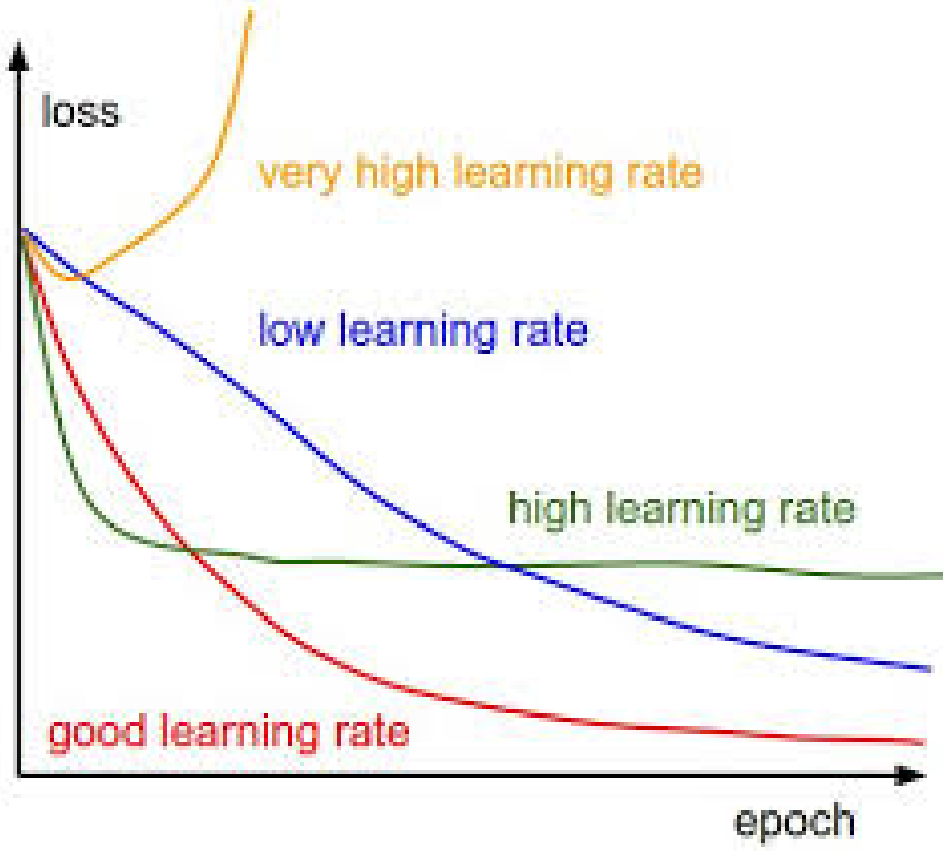
- A visual explanation of optimisation algorithms with a visualisation tool:

https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c



Faster optimisers

Conclusion


loss
very high learning rate
low learning rate
high learning rate
good learning rate
epoch

STILL WAITING
FOR MY NEURAL NET TO COMPLETE TRAINING